Examen DeliverUS - Modelo A - Marzo (Reviews)

Se ha incorporado un nuevo requisito en la aplicación DeliverUS, que permite a los clientes dejar reviews sobre los restaurantes en los que han realizado pedidos. Además se podrá consultar la valoración media que los clientes tienen sobre los restaurantes. En DeliverUS, las reviews representan la valoración de los clientes sobre un restaurante.

Requisitos funcionales a implementar

RF1. Listado de reviews de un restaurante: Como usuario, quiero listar las reviews de un restaurante, para ver las valoraciones de otros clientes.
`GET /restaurants/:restaurantId/reviews`

RF2. Creación de una review: Como cliente, quiero dejar una review en un restaurante en el que haya pedido, para valorar mi experiencia.
`POST /restaurants/:restaurantId/reviews`

RF3. Edición de una review: Como cliente, quiero poder editar mi review, para corregir o actualizar mi opinión.
`PUT /restaurants/:restaurantId/reviews/:reviewId`

RF4. Eliminación de una review: Como cliente, quiero poder eliminar mi review, para retirar mi opinión si ya no es relevante.
`DELETE /restaurants/:restaurantId/reviews/:reviewId`

RF5. Cálculo de la media de estrellas de un restaurante: Como usuario, quiero ver la media de puntuaciones de un restaurante, para conocer su valoración general.
El atributo virtual avgStars debe calcularse como el promedio de todas las reviews. El objeto json que representa al restaurante tendrá la forma siguiente (nótese que se incluye la propiedad avgStars). Si el restaurante no tiene reviews, avgStars debe ser null o undefined.

(1) Preparación del entorno: `npm run install:all:win`

(2) Ejecutar el backend: `npm run start:backend`

(3) Comprobar que todo está bien: `npm run test:backend`

1. Migraciones, modelos y cambios necesarios (2 puntos)

Cree y modifique las migraciones necesarias para implementar el modelado conceptual así como cree y modifique los modelos necesarios. Complete los ficheros /src/database/migrations/20250723214455-create-review.js y /src/models/Review.js, además de las modificaciones necesarias sobre otros ficheros.

Migraciones de review:

```
id: {
   allowNull: false,
   autoIncrement: true,
   primaryKey: true,
   type: Sequelize.INTEGER
 },
 stars: {
   allowNull:false,
   type: DataTypes.INT
 },
 body: {
   allowNull:true,
   type: DataTypes.TEXT
 },
 restaurantId: {
   allowNull: false,
   type: Sequelize.INTEGER,
   onDelete: 'CASCADE', // Se borran las reseñas de los restaurantes eliminados
   references: {
     model: {
       tableName: 'Restaurants'
     },
     key: 'id'
   }
 },
customerId: {
  allowNull: false,
  type: Sequelize.INTEGER,
  onDelete: 'CASCADE', // Se borran las reseñas de los usuarios eliminados
  references: {
    model: {
      tableName: 'Users'
    },
    key: 'id'
  }
}
createdAt: {
  allowNull: false,
  type: Sequelize.DATE,
  defaultValue: new Date()
},
updatedAt: {
```

```
    allowNull: false,
    type: Sequelize.DATE,
    defaultValue: new Date()
  }
```

Modelo de review:

Definimos las claves ajenas:

```
Review.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })
Review.belongsTo(models.User, { foreignKey: 'customerId', as: 'user' })
```

Definimos la tabla review:

```
 Review.init({
    stars: {
      allowNull:false,
      type: DataTypes.INT
    },
    body: {
      allowNull:true,
      type: DataTypes.TEXT
    },
    restaurantId: {
      allowNull: false,
      type: Sequelize.INTEGER,
      onDelete: 'CASCADE',
      references: {
        model: {
          tableName: 'Restaurants'
        },
        key: 'id'
      }
    },
    customerId: {
      allowNull: false,
      type: Sequelize.INTEGER,
      onDelete: 'CASCADE',
      references: {
        model: {
          tableName: 'Users'
        },
        key: 'id'
      }
    }
  }
```

Añadimos la clave ajena en el modelo de restaurante (/src/models/Restaurant.js):

```
Restaurant.hasMany(models.Review, { foreignKey: 'restaurantId', as: 'reviews' })
```

Añadimos la clave ajena en el modelo de usuario (/src/models/User.js):

```
User.hasMany(models.Review, { foreignKey: 'customerId', as: 'reviews' })
```

2. Rutas de Review (1 punto)

Implemente las siguientes rutas:

```
RF1: GET /restaurants/:restaurantId/reviews
RF2: POST /restaurants/:restaurantId/reviews
RF3: PUT /restaurants/:restaurantId/reviews/:reviewId
RF4: DELETE /restaurants/:restaurantId/reviews/:reviewId
```

Se le proporciona el fichero /src/routes/ReviewRoutes.js para definir estas rutas y recuerde incorporar los middlewares necesarios en cada ruta.

Rutas de review (ir mirando los requisitos funcionales necesarios para cada ruta, en ReviewMiddleware.js se encuentran definidas las funciones para verificar los requisitos):

```
const loadReviewRoutes = function (app) {
    app.route('/restaurants/:restaurantId/reviews')
    .get(
      checkEntityExists(Restaurant, 'restaurantId')),
      ReviewController.index
    .post(
      isLoggedIn,
      hasRole('customer'),
       checkEntityExists(Restaurant, 'restaurantId'),
       userHasPlacedOrderInRestaurant,
      checkCustomerHasNotReviewed,
      ReviewValidation.create,
      handleValidation,
      ReviewController.create
    )

    app.route('/restaurants/:restaurantId/reviews/:reviewId ')
    .put(
      isLoggedIn,
      hasRole('customer'),
      checkReviewOwnership,
      checkEntityExists(Restaurant, 'restaurantId'),
      checkEntityExists(Review, 'reviewId'),
      checkReviewBelongsToRestaurant,
      ReviewValidation.create,
      handleValidation,
      ReviewController.update
    )
.delete(
  isLoggedIn,
  hasRole('customer'),
  checkReviewOwnership,
  checkEntityExists(Restaurant, 'restaurantId'),
  checkEntityExists(Review, 'reviewId'),
  checkReviewBelongsToRestaurant,
  ReviewController.destroy
)
}
```

3. Validaciones para Review (1 puntos)

Implemente las reglas de validación para la creación y la edición de una review. Se le proporciona el fichero /src/controllers/Validation/ReviewValidation.js para definir estas validaciones.

```
const create = [
   check('stars').exists().isInt({ min: 0, max: 5 }),
    check('body').optional({nullable:  true,  checkFalsy:  true}).isString().isLength({
min: 1, max: 255 }).trim()
]
const update = [
     check('stars').exists().isInt({ min: 0, max: 5 }),
     check('body').optional({nullable:  true,  checkFalsy:  true}).isString().isLength({
min: 1, max: 255 }).trim()
]
```

4. Controlador de Review (2 puntos)

Implemente las funciones necesarias para RF1, RF2, RF3 y RF4. Se le proporciona el fichero /src/controllers/ReviewController.js para definir estas funciones.

```
const ReviewController = {
  // Controller para listar las reviews (GET)
    async index (req, res) {
      try{
        const reviews = await Review.findAll({
          where: { restaurantId: req.params.restaurantId },
          order: [['createdAt', 'DESC']]
          }
        )
        res.json(reviews)
      }
      catch(err) {
      res.status(500).send(err)
      }
    },

  // Controller para publicar una nueva review (POST)
  async create (req, res) {
    try{
      const newReview = Review.build(req.body)
      newReview.customerId = req.user.id
      newReview.restaurantId = req.param.restaurantId
      const createdReview = await newReview.save()
      res.json(createdReview)
    }
      catch(err) {
      res.status(500).send(err)
    }
  },
```

```
  // Controller para actualizar una review (UPDATE)
  async update (req, res) {
    try{
      const currentReview = await Review.findByPk(req.params.reviewId)
      currentReview.stars = req.body.stars
      currentReview.body = req.body.bodyconst
      res.json(currentReview)
    }
    catch(err) {
      res.status(500).send(err)
    }
  },

  // Controller para eliminar una review (DELETE), siempre igual en todos!!
  async destroy (req, res) {
    try {
      const result = await Review.destroy({ where: { id: req.params.reviewId } })
      let message = ''
      if (result === 1) {
        message = 'Sucessfuly deleted review id.' + req.params.reviewtId
      } else {
        message = 'Could not delete review.'
      }
      res.json(message)
    }
    catch (err) {
      res.status(500).send(err)
    }
  }

}
```

5. Comprobación de que el cliente puede crear una review (2 puntos)

Implemente las dos comprobaciones que un cliente debe cumplir para publicar una review, esto es:

- Tiene al menos un pedido en el restaurante en el que quiere poner una review.
- No tiene ya una review publicada para ese restaurante.

Se le proporciona el fichero /src/middlewares/ReviewMiddleware.js con los prototipos para sendas funciones userHasPlacedOrderInRestaurant, checkCustomerHasNotReviewed que ha de implementar.

```
const userHasPlacedOrderInRestaurant = async (req, res, next) => {
  try {
    const userId = req.user.id
    const restaurantId = req.params.restaurantId
    const order = await Order.findOne({where: { userId,restaurantId }})
    if (!order) {
      return res.status(409).json({
        message: 'You must have placed an order in this restaurant to write a review.'
      })
```

```
    }
    next()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
}

const checkCustomerHasNotReviewed = async (req, res, next) => {
  try {
    const userId = req.user.id
    const restaurantId = req.params.restaurantId
    const review = await Review.findOne({ where: {customerId: userId,restaurantId} })
    if (review) {
      return res.status(409).json({
        message: 'You have already reviewed this restaurant.'
      })
    }
    next()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
}
```

6. Cálculo de avgStars (2 puntos)

Implemente la lógica para calcular el atributo virtual avgStars en la función getAvgStars que encontrará en src/models/Restaurant.js. Sólo es necesario completar esta función para que el modelo incluya la propiedad virtual, el resto de cambios necesarios ya están incorporados.

```
async getAvgStars () {
   if (!this.reviews || this.reviews.length === 0) {
        return 0
   }
   const totalStars = this.reviews.reduce((sum, review) =>
           {return sum + review.stars}, 0)
   return totalStars / this.reviews.length
}
```

FUNCIONES:

5. Comprobaciones de seguridad y consistencia (2 puntos)

Implemente las comprobaciones necesarias para asegurar que un usuario no accede a direcciones de envío de otro. Se le proporciona el fichero /src/middlewares/AddressMiddleware.js para definir esta lógica.

```
export const checkShippingAddressOwnership = async (req, res, next) => {
  try {
    const address = await ShippingAddress.findByPk(req.params.shippingAddressId)
    if (address === null){
      throw new Error("The shipping address does not exist");
    }
    if (address.userId !== req.user.id){
      return res.status(403).json({ message: 'Forbidden' })
    }
    next()
  }
  catch (err) {
    res.status(500).send(err)
  }
}
```

6. Validación de creación de Producto (1 punto)

Valide la creación de producto teniendo en cuenta la relación definida en el modelo conceptual y el RF6. Se le proporciona el fichero /src/controllers/validation/ProductValidation.js con el prototipo de la función checkScheduleBelongsToRestaurantOnCreate.

```
const checkScheduleBelongsToRestaurantOnCreate = async (value, { req }) => {
  try {
    const schedule = await Schedule.findByPk(value)
    if (schedule === null) {
      throw new Error("The schedule does not exist");
    }
    if (schedule.restaurantId !== req.body.restaurantId){
      throw new Error("The schedule is not from the restaurant of the product");
    }
    next()
  }
  catch (err) {
    return res.status(500).send(err)
  }
}
```

7. Validación de edición de Producto (1,5 puntos)

Valide la edición de producto teniendo en cuenta la relación definida en el modelo conceptual y el RF7. Se le proporciona el fichero /src/controllers/validation/ProductValidation.js con el prototipo de la función checkScheduleBelongsToRestaurantOnUpdate.

```
const checkScheduleBelongsToRestaurantOnUpdate = async (value, { req }) => {
  try {
    const schedule = await Schedule.findByPk(value)
    if (schedule === null) {
    throw new Error('The schedule does not exist')
    }
    const product = await Product.findByPk(req.params.productId)
    if (product === null) {
      throw new Error('The product does not exist')
    }
    if (schedule.restaurantId !== product.restaurantId) {
      throw new Error('The schedule is not from the restaurant of the product')
    }
    next()
  }
  catch (err) {
    return res.status(500).send(err)
  }
}
```

8. Detalle de restaurante incluyendo sólo productos activos (1,5 puntos)

Implemente la siguiente ruta:

`RF5: GET /restaurants/:restaurantId/showWithActiveProducts`

Implemente asimismo la función showWithActiveProducts cuyo prototipo se le proporciona en el fichero /src/controllers/RestaurantController.js para cumplir el RF5. Productos con scheduleId nulo NO se consideran activos automáticamente. La consulta debe considerar la hora actual del servidor. Puede usar new Date().toTimeString().split(' ')[0] para obtener el tiempo actual en formato HH:mm:ss.

```
// Un producto es activo si:
// - Tiene scheduleId NO nulo
// - La hora actual está dentro de su horario: startTime <= ahora <= endTime
const showWithActiveProducts = async function (req, res) {
  try {
    const currentTime = new Date().toTimeString().split(' ')[0]
    const restaurant = await Restaurant.findByPk(req.params.restaurantId, {
      include: [
        {
          model: Product,
          as: 'products',
          required: false,
          where: { scheduleId: { [Op.ne]: null } },
          include: [
            {
              model: Schedule,
              as: 'schedule',
              required: true,
              where: {
                startTime: { [Op.lte]: currentTime },
                endTime: { [Op.gte]: currentTime }
              }
            }
          ]
        }
      ]
    })
    if (!restaurant) {
      return res.status(404).json({ message: 'Restaurant not found' })
    }
    res.json(restaurant)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}
```